

System Design

System Architecture Basics

Part 1: What is System

- What is System and Real Life Example
- Software System Example: Facebook Login
- What is the Difference Between Processing and Component?

Part 2: System Core Components

- Input
- Processing
- Output
- Storage

Part 3: What is Component?

- What is Component?
- Components in Software System
- Why are Components Separated?
 - Separation of Concerns
 - Scalability
 - Maintainability
 - Team Collaboration

Part 4: How do Components Communicate?

- API Calls
- Message Queue
- Shared Database
- Event-Driven

Part 5: How to Apply System Thinking in HLD Design?

- Step 1: Identify System from Requirements
- Step 2: Identify Components
- Step 3: Communication Design
- Step 4: Data Flow

Part 6: Diagrams & Tools

6.1 Why Diagrams Matter in System Design

- A system design is incomplete without diagrams
- Failing to draw diagrams in an interview almost guarantees a failed interview
- Communication tool for teams

6.2 UML Diagrams for System Design

- Sequence Diagram — request flow দেখাও ★★★★★★
- Component Diagram — HLD-এর জন্য ★★★★★★
- Class Diagram — LLD-এর জন্য ★★★★★★
- Data Flow Diagram — data কোথায় যায় ★★★★★★

6.3 Diagramming Tools

- Excalidraw — interview standard ★★★★★★
- draw.io / Lucidchart — professional use ★★★★★★
- Mermaid — code-based diagrams ★★★★★★

6.4 How to Draw Diagrams in Interviews

- কী আগে আঁকবে, কী পরে ★★★★★★
- Detail কতটুকু দেবে ★★★★★★
- Iterate করার strategy ★★★★★★

Part 7: Common Questions and Answers

- Where to Start?
- How Many Components Should I Create?
- What is the Biggest Mistake in System Design?

Networking Fundamentals

- Network & Internet - System Design Perspective
- Packet with System Design Perspective
- Communication Protocols Comparison
- DNS Deep Dive (SD perspective)
- OSI Model (SD perspective)
- TCP vs UDP (SD perspective)
- IP Addresses (SD perspective)
- Checksums
- **Latency Deep Dive**
 - Network Latency Optimization (TCP Connection Pooling, SSL Session Caching, Data Compression)
 - Memory Access Latency (Heap Management, GC Tuning)
 - Disk I/O Latency Optimization (Sequential IO, Page Cache, Zero Copy)
 - CPU Latency & Context Switching

System Design Fundamentals

- What happens when you enter Google.com?
- Latency Numbers Every Engineer Should Know ★★★★★★
- Back-of-the-envelope Estimation ★★★★★★
- SLA, SLO, SLI ★★★★★★
- Scalability Principles ★★★★★★
- Vertical vs Horizontal Scaling ★★★★★★
- Availability vs Consistency and trades0-offs (CAP Theorem) ★★★★★★
- PACELC Theorem ★★★★★★
- Partition Tolerance ★★★★★★
- ACID Properties ★★★★★★
- BASE Properties (Eventually Consistent) ★★★★★★
- Single Point of Failure (SPOF) ★★★★★★
- Fault Tolerance & Resilience ★★★★★★
- Two Generals Problem & Byzantine Fault Tolerance ★★★★★★
- Idempotency in Distributed Systems ★★★★★★
- Vector Clocks & Logical Clocks ★★★★★★
- Disaster Recovery Planning ★★★★★★
- System Reliability Engineering (SRE) ★★★★★★
- Decentralization Principle in Scalability ★★★★★★
- Independence & Loose Coupling Principle ★★★★★★
- Modularity as Foundation for Scalability ★★★★★★
- Stateless vs Stateful Architecture ★★★★★★
- Change Data Capture (CDC) ★★★★★★
- Asynchronous APIs ★★★★★★
- Consensus Algorithms (Raft Paxos, PBFT) ★★★★★★
- Gossip Protocol ★★★★★★

ID & Key Generation Strategies

- Base62 Encoding ★★★★★★
- MD5 Hash Approach for ID Generation ★★★★★★
- Counter Approach ★★★★★★
- Snowflake ID (Twitter) ★★★★★★
- ULID (Universally Unique Lexicographically Sortable) ★★★★★★
- Distributed Counter with Zookeeper ★★★★★★
- Key Generation Service (KGS) ★★★★★★
- KGS — Concurrent Access Handling (Two-table Strategy) ★★★★★★
- KGS — In-memory Key Caching ★★★★★★

Load Balancing & Distribution

- Load Balancer Types (Layer 4 vs Layer 7) ★★★★★★
- Load Balancing Algorithms ★★★★★★
- Power of Two Choices ★★★★★★
- Round Robin ★★★★★★
- Weighted Round Robin ★★★★★★
- Least Connections ★★★★★★
- IP Hash ★★★★★★
- DNS-base load balancers
- Geographic Load Balancing ★★★★★★
- Anycast Load Balancing ★★★★★★
- Reverse Proxy vs Forward Proxy ★★★★★★
- CDN (Content Delivery Network) ★★★★★★
- Edge Computing ★★★★★★
- Auto Scaling Strategies ★★★★★★ (Deep dive: Section 28 Cloud/DevOps)
- Content Compression at Load Balancer ★★★★★★
- Rate Limiting & Traffic Shaping at Load Balancer ★★★★★★
- Load Balancer Redundancy & Failover ★★★★★★
- Load Balancer Logging & Monitoring ★★★★★★
- Load Balancer Integration with Auto-Scaling ★★★★★★
- Hardware Load Balancer vs Software Load Balancer ★★★★★★
- Global Server Load Balancing (GSLB) ★★★★★★

Caching Strategies

Cache Patterns

- Cache-Aside (Lazy Loading) ★★★★★★
- Read-Through Cache ★★★★★★
- Write-Through ★★★★★★
- Write-Behind (Write-Back) ★★★★★★
- Write-Around Cache ★★★★★★
- Refresh-Ahead ★★★★★★
- Pareto Principle (80/20 Rule) in Caching ★★★★★★

Cache Levels

- Browser Cache ★★★★★★
- CDN Cache ★★★★★★
- Reverse Proxy Cache ★★★★★★
- Application Cache ★★★★★★
- Database Cache ★★★★★★
- Multi-level Cache Hierarchy ★★★★★★

Cache Management

- Cache Invalidation Strategies ★★★★★★
- Cache Consistency Models ★★★★★★
- TTL (Time-To-Live) Expiration ★★★★★★
- Stale-While-Revalidate (SWR) ★★★★★★
- Distributed Caching (Redis Cluster) ★★★★★★
- Cache Partitioning ★★★★★★
- Hot Key Problem ★★★★★★
- Cache Warming Strategies ★★★★★★
- Cache Stampede Prevention (Thundering Herd) ★★★★★★
- Probabilistic Early Expiration ★★★★★★
- Cache Penetration — Null Value Caching ★★★★★★
- Cache Pollution & LRU Eviction ★★★★★★
- Connection Pooling for Latency Reduction ★★★★★★
- Big Key Problem ★★★★★★

Cache Eviction Policies

- LRU — Least Recently Used ★★★★★★
- LFU — Least Frequently Used ★★★★★★
- FIFO — First In First Out ★★★★★★
- Random Eviction ★★★★★

Database Architecture

- SQL vs NoSQL Trade-offs ★★★★★★
- NewSQL Databases (CockroachDB, TiDB) ★★★★★★
- Database Scaling Patterns ★★★★★★
- Read/write Replicas ★★★★★★
- Data Replication vs Component Replication ★★★★★★
- Stateless Replication ★★★★★★
- Stateful Replication ★★★★★★
- Sticky Session / Session Affinity ★★★★★★
- Session Clustering ★★★★★★
- Master-Slave Replication ★★★★★★
- Master-Master Replication ★★★★★★
- Multi-Region Active-Active ★★★★★★
- Raft Consensus Algorithm ★★★★★★
- Synchronous Replication vs Asynchronous Replication ★★★★★★
- Database Sharding ★★★★★★
- Federation ★★★★★★
- ACID Transactions ★★★★★★
- Distributed Transactions (2PC, Saga Pattern) ★★★★★★
- Database Per Service Pattern ★★★★★★
- Database Connection Pooling ★★★★★★
- Distributed Locking (Redis/Zookeeper) ★★★★★★

- Database Connection Pooling Internals (PgBouncer, HikariCP) ★★★★★

Data Partitioning Strategies

- Horizontal Partitioning (Sharding) ★★★★★
- Vertical Partitioning ★★★★★
- Range-based Partitioning ★★★★★
- Hash-based Partitioning ★★★★★
- Directory-based Partitioning ★★★★★
- Consistent Hashing ★★★★★

Data Management

- Data Consistency Patterns ★★★★★
- Database Indexing Strategies ★★★★★
- Query Optimization ★★★★★
- Database Monitoring & Profiling ★★★★★

Database Maintenance

- Active Cleanup (Cron Job / Scheduled Cleanup) ★★★★★
- Passive Cleanup / Lazy Deletion ★★★★★

Storage Systems

- Object Storage (S3) ★★★★★
- Block Storage vs File Storage ★★★★★
- LSM Tree vs B-Tree Storage ★★★★★
- Search Engines (Elasticsearch) ★★★★★
- Vector Databases (Pinecone, Weaviate, Qdrant) ★★★★★
- Time Series Databases ★★★★★
- Bloom Filter ★★★★★

API Design at Scale

- API Gateway Deep Dive ★★★★★
- API Gateway — Request & Response Transformation ★★★★★
- API Gateway — Distributed Tracing ★★★★★
- API Gateway as Load Balancer + Reverse Proxy ★★★★★
- API Gateway — Ingress Controller for Kubernetes ★★★★★
- API Gateway — Service Mesh Integration ★★★★★
- Rate Limiting at Scale ★★★★★
- API Versioning Strategy ★★★★★
- API Key for Rate Limiting & Abuse Prevention ★★★★★
- GraphQL vs REST at Scale ★★★★★

- gRPC & Protocol Buffers ★★★★★
- Long Polling vs WebSocket vs SSE ★★★★★
- Webhook Architecture ★★★★★

Microservices Architecture

- Microservices vs Monolithic ★★★★★
- Service Decomposition Strategies ★★★★★
- Domain-Driven Design (DDD) ★★★★★
- Bounded Contexts ★★★★★
- Strangler Pattern (Incremental Migration) ★★★★★
- Polyglot Architecture ★★★★★
- Private Data Ownership per Service ★★★★★
- Microservice Team Ownership & Organizational Alignment ★★★★★
- Service Versioning Strategy ★★★★★
- API Contract Testing (Pact) ★★★★★

Service Communication

- Synchronous (HTTP/REST, gRPC) ★★★★★
- Asynchronous (Message Queues) ★★★★★
- Service Discovery ★★★★★
- API Gateway Pattern ★★★★★
- Circuit Breaker Pattern ★★★★★
- Bulkhead Pattern ★★★★★
- Retry Mechanisms & Exponential Backoff ★★★★★
- Idempotency in Microservices ★★★★★
- Service Mesh (Istio, Linkerd) ★★★★★ → *(Deep dive: Section 28 Cloud/DevOps)*
- Service Mesh vs API Gateway ★★★★★
- Distributed Tracing ★★★★★
- Container Orchestration (Kubernetes) → *(Deep dive: Section 29 Docker)*
- Service-to-Service Authentication ★★★★★
- Data Management in Microservices ★★★★★
- Microservices Testing Strategies ★★★★★
- GraphQL for Inter-Service Communication ★★★★★
- Sidecar Pattern ★★★★★

Message Queues & Event-Driven Architecture

Message Queue Patterns

- Point-to-Point ★★★★★
- Publish-Subscribe ★★★★★
- Request-Reply ★★★★★

Message Queue Systems

- RabbitMQ ★★★★★
- Apache Kafka ★★★★★
- Kafka Architecture Deep Dive ★★★★★
- Amazon SQS ★★★★★
- Redis Pub/Sub ★★★★★
- Database as a Message Queue ★★★★★

Event-Driven Patterns

- Event Sourcing ★★★★★
- CQRS (Command Query Responsibility Segregation) ★★★★★
- Saga Pattern for Distributed Transactions ★★★★★
- Event-Driven Microservices ★★★★★
- Message Ordering & Delivery Guarantees ★★★★★
- At-least-once vs At-most-once vs Exactly-once ★★★★★
- Dead Letter Queues ★★★★★
- Message Deduplication ★★★★★
- Stream Processing (Apache Kafka Streams) ★★★★★
- Event Schema Registry (Confluent) ★★★★★

Security Architecture

- Authentication vs Authorization ★★★★★
- Identity and Access Management (IAM) ★★★★★
- Zero Trust Architecture ★★★★★
- OAuth 2.0 & OpenID Connect ★★★★★
- JWT Security Best Practices ★★★★★
- API Security Patterns ★★★★★
- Rate Limiting & DDoS Protection ★★★★★
- Encryption at Rest & in Transit ★★★★★
- Certificate Management ★★★★★
- Network Security → *(Reference: Section 24 web security)* ★★★★★
- Security Monitoring & Incident Response ★★★★★
- Digital Signature ★★★★★
- Digital Certificate & Components ★★★★★
- Common Hashing Algorithms (MD5, SHA-1, SHA-256, SHA-3) ★★★★★
- TLS — Pre-master Secret & Master Secret Derivation ★★★★★
- Site-to-Site VPN vs Remote Access VPN vs SSL VPN ★★★★★
- VPN Tunneling Protocols (IPsec, OpenVPN) ★★★★★
- Network Segmentation & DMZ (Demilitarized Zone) ★★★★★
- Firewall Rules & Traffic Control ★★★★★
- mTLS (Mutual TLS) ★★★★★ *(Reference: Section 31 Networking)*

Reliability & Fault Tolerance

Reliability Fundamentals

- Reliability vs Availability vs Fault Tolerance ★★★★★
- Time-Based Availability ★★★★★
- Request-Based Availability ★★★★★
- The Nines of Availability (99.9%, 99.99%, 99.999%, 99.9999%) ★★★★★

Fault Tolerance Design

- Redundancy (Active / Passive / Cold) ★★★★★
- Active Redundancy (Hot Spare) ★★★★★
- Passive Redundancy (Warm Spare) ★★★★★
- Cold Redundancy (Backup) ★★★★★
- Fault Models (Response, Timeout, Crash, Arbitrary) ★★★★★
- Partial Failures & Cascading Failures ★★★★★
- Graceful Degradation vs Fail Fast ★★★★★

Health Checks & Monitoring

- Ping-Based Health Check ★★★★★
- Heartbeat-Based Health Check ★★★★★
- External Monitoring Service ★★★★★

Recovery Strategies

- Stateless Recovery (Hot/Warm Standby) ★★★★★
- Stateful Recovery (DB Failover, Cache Failover) ★★★★★
- Load Balancer Redundancy & Virtual IP Failover ★★★★★
- Datacenter Redundancy ★★★★★

System Stability Patterns

- Timeouts ★★★★★
- Retries with Exponential Backoff ★★★★★
- Circuit Breaker Pattern (Closed/Open/Half-Open) ★★★★★

High Availability & Disaster Recovery

- Availability Patterns (99.9%, 99.99%, 99.999%) ★★★★★
- Redundancy Strategies ★★★★★
- Geographic Distribution ★★★★★
- Multi-Region Deployments ★★★★★
- Backup Strategies ★★★★★
- Recovery Time Objective (RTO) ★★★★★

- Recovery Point Objective (RPO) ★★★★★★
- Chaos Engineering ★★★★★★
- Circuit Breaker Implementation ★★★★★★
- Bulkhead Isolation ★★★★★★
- Blue-Green Deployment ★★★★★★ → (Deep dive: Section 30 CI/CD)
- Canary Deployment ★★★★★★ → (Deep dive: Section 28 Cloud/DevOps)
- Rolling Deployment ★★★★★★ → (Deep dive: Section 28 Cloud/DevOps)

Performance & Scalability

- Performance Metrics & SLAs ★★★★★★
- Latency vs Throughput ★★★★★★
- Little's Law ★★★★★★
- Bottleneck Identification ★★★★★★
- Performance Testing Strategies ★★★★★★
- Capacity Planning ★★★★★★
- USL (Universal Scalability Law) ★★★★★★
- Resource Utilization Optimization ★★★★★★
- Database Performance Tuning ★★★★★★
- API Performance Optimization ★★★★★★
- Frontend Performance Architecture ★★★★★★
- Edge Computing Strategies ★★★★★★
- Real-time vs Batch Processing ★★★★★★
- Stream Processing Architecture ★★★★★★
- Tail Latency (P99, P99.9 percentile) ★★★★★★
- Queue Build-up Analysis ★★★★★★
- Serial vs Concurrent Request Latency ★★★★★★
- Cost Optimization ★★★★★★
- Backpressure Handling ★★★★★★

System Design Tradeoffs

Scaling

- Vertical vs Horizontal Scaling ★★★★★★
- Scalability vs Performance ★★★★★★

Consistency & Availability

- Consistency vs Availability (CAP) ★★★★★★
- Strong vs Eventual Consistency ★★★★★★
- ACID vs BASE ★★★★★★

Data & Storage

- SQL vs NoSQL ★★★★★
- Normalization vs Denormalization ★★★★★
- Read-Through vs Write-Through Cache ★★★★★
- In-Memory vs Disk Storage ★★★★★

Communication & Processing

- Synchronous vs Asynchronous Communication ★★★★★
- Batch vs Stream Processing ★★★★★
- Long Polling vs WebSockets vs SSE ★★★★★
- Push vs Pull Architecture ★★★★★
- REST vs GraphQL vs gRPC ★★★★★
- REST vs RPC ★★★★★

Architecture

- Monolith vs Microservices ★★★★★
- Stateful vs Stateless Design ★★★★★
- Concurrency vs Parallelism ★★★★★

Performance

- Latency vs Throughput ★★★★★
- Performance vs Cost ★★★★★

Monitoring & Observability

- Application Performance Monitoring (APM) ★★★★★
- Distributed Tracing (Jaeger, Zipkin) ★★★★★
- Centralized Logging ★★★★★
- Log Aggregation (ELK Stack) ★★★★★
- Structured Logging ★★★★★
- Log Correlation ★★★★★
- Apache Spark for Real-time Analytics ★★★★★
- Trending Topics with Batch Job & Ranking ★★★★★
- OpenTelemetry ★★★★★

Metrics & Monitoring

- System Metrics (CPU, Memory, Disk, Network) ★★★★★
- Application Metrics (Response Time, Error Rate, Throughput) ★★★★★
- RED Method (Rate, Errors, Duration) ★★★★★
- USE Method (Utilization, Saturation, Errors) ★★★★★
- Business Metrics (Conversion Rate, Revenue) ★★★★★
- Alerting Strategies ★★★★★

- Incident Management ★★★★★
- Root Cause Analysis ★★★★★
- Performance Debugging ★★★★★
- Health Checks & Status Pages ★★★★★

PRACTICE & REAL SYSTEMS

Design Thinking Framework ★★★★★

- Requirement Clarification
- Bottleneck Identification
- Data & Control Flow
- Stateless vs Stateful
- Failure Handling

Must-Know Design Patterns

- Rate Limiting (Token Bucket, Leaky Bucket) ★★★★★
- Circuit Breaker & Retry Logic ★★★★★
- Leader Election ★★★★★
- Health Checks & Heartbeats ★★★★★
- CQRS & Event Sourcing ★★★★★

Domain-Specific Design Patterns

Video Processing Pipeline

- File Chunking (Scene-based vs Timestamp-based) ★★★★★
- Content Filter (Copyright, Piracy, NSFW) ★★★★★
- Transcoding (FFmpeg, AWS Elemental MediaConvert) ★★★★★
- Quality Conversion (4K, 1080p, 720p) ★★★★★
- Subtitle & Thumbnail Generation Pipeline ★★★★★

Streaming Architecture

- Adaptive Bitrate Streaming (HLS) ★★★★★
- Netflix Open Connect Program ★★★★★
- Resume Playback with Offset ★★★★★

Geo & Recommendation

- Geo-blocking Implementation ★★★★★
- Geolocation Routing (Amazon Route53) ★★★★★
- Collaborative Filtering (Recommendation) ★★★★★
- Netflix Recommendation Engine Approach ★★★★★

Graph & Social Systems

- Social Graph Design 
- Graph Database (Neo4j, ArangoDB) 
- Mutual Friends / Recommendation Algorithm 

Real-World Practice Problems

Easy

- Design a Rate Limiter
- Design a URL Shortener (TinyURL)
- Design a Notification System
- Design an API Gateway
- Design a Distributed Cache System (Redis)
- Design Load Balancer
- Design Authentication System
- Design UPI (Unified Payments Interface)
- Design Parking Garage
- Design Vending Machine

Medium

- Design a Social Media Feed (Twitter/Facebook)
- Design Instagram Reels / YouTube System
- Design a Chat System (WhatsApp/Slack)
- Design a Music Streaming Service (Spotify)
- Design an E-commerce System (Amazon)
- Design a Payment System (PayPal/Stripe)
- Design a Food Delivery System (Uber Eats)
- Design a Distributed Job Scheduler
- Design a File Storage System (Dropbox/Google Drive)
- Design a Web Crawler
- Design a Distributed Message Queue (Kafka)
- Design Tinder
- Design Reddit
- Design TikTok
- Design Shopify
- Design Airbnb
- Design Flight Booking System
- Design Online Code Editor
- Design Digital Wallet
- Design an Analytics Platform (Metrics & Logging)

Hard

- Design a Video Streaming Platform (YouTube/Netflix)
- Design a Ride-sharing Service (Uber/Lyft)
- Design a Search Engine (Google)
- Design a Stock Trading System
- Design a Collaboration Tool (Google Docs)
- Design a Video Conferencing System (Zoom)
- Design a Multi-tenant SaaS Platform
- Design an AI Chatbot System
- Design Location Based Service (Yelp)
- Design Google Maps
- Design Ticket Booking System (BookMyShow)
- Design Code Deployment System
- Design Distributed Cloud Storage (S3)
- Design Distributed Locking Service

System Design Interview Framework

RESHADED Framework ★★★★★

- R: Requirements Clarification (5-7 min)
- E: Estimation & Capacity (3-5 min)
- S: System APIs (3-5 min)
- H: High Level Design (10-15 min)
- A: Articulate Data Model (5 min)
- D: Deep Dive (10-15 min)
- E: Evaluate Trade-offs (3-5 min)
- D: Done & Extensions (2-3 min)

R — Requirements Clarification ★★★★★

Functional Requirements ★★★★★

- What will the system do? (core features)
- What actions can the user take?
- What data will be stored and retrieved?
- What are the business rules?
- What integrations are needed? (payment, email, SMS)
- What happens in error cases?

Non-Functional Requirements ★★★★★

Performance ★★★★★

- API response time target? (e.g. < 200ms)
- Peak concurrent users?

- Read/Write ratio? (e.g. 100:1)
- Throughput target? (requests per second)

Scalability ★★★★★

- Starting users vs growth target?
- Traffic spikes expected?
- Geographic distribution needed?
- Horizontal scaling required?

Availability & Reliability ★★★★★

- Uptime requirement? (99.9% / 99.99% / 99.999%)
- Is planned downtime acceptable?
- Recovery Point Objective (RPO)?
- Recovery Time Objective (RTO)?

Security ★★★★★

- Authentication type? (JWT, OAuth, Session)
- Authorization granularity? (RBAC / ABAC)
- Encryption needed? (at rest / in transit)
- Compliance required? (GDPR, HIPAA, SOC2)
- Rate limiting needed?

Data ★★★★★

- Data retention period?
- Backup strategy?
- Consistency model? (Strong vs Eventual)
- Data volume? (GB / TB / PB)

Maintainability ★★★★★

- Logging & monitoring needed? ★★★★★
- Deployment frequency? ★★★★★
- Observability requirements? ★★★★★

Cost ★★★★★

- Cloud budget constraints?
- Cost optimization required?

Extended Requirements ★★★★★

- Explicitly define Out of Scope ★★★★★
- State CAP position (Availability vs Consistency) upfront ★★★★★

- Analytics & Reporting ★★★★★
- Admin Dashboard ★★★★★
- Audit Logging ★★★★★
- A/B Testing ★★★★★
- Multi-language Support ★★★★★
- Multi-currency Support ★★★★★
- Third-party Integrations (Slack, Zapier) ★★★★★
- Content Moderation Dashboard ★★★★★
- Data Export Feature ★★★★★
- White-labeling ★★★★★

E — Estimation & Constraints ★★★★★

- Traffic Estimation (Read/Write Ratio)
- Bandwidth Estimation
- Storage Estimation
- Cache Estimation (80/20 Rule)
- High Level Estimation Summary Table
- Estimation Completeness Checklist

S — API Design ★★★★★

- REST API Design for SD Interview
- API Key & Abuse Prevention

H — High Level Design (HLD) ★★★★★

- HLD vs LLD — Difference & When to Use
- Component Identification
- Monolithic vs Microservices Decision
- High Level Architecture Diagram

A — Articulate Data Model ★★★★★

- Entity Identification
- Relationship Mapping
- SQL vs NoSQL Decision Framework
- Database Schema Design

D — Deep Dive ★★★★★

- Detailed Component Deep Dive
- Caching Strategy (Redis, Eviction Policy, Cache Miss)
- Data Partitioning Strategy (Sharding, Consistent Hash)
- Database Cleanup Strategy (Active vs Passive)

- Metrics & Analytics Collection
- Security (Private URLs, Authorization, API Gateway)
- Feed Generation (Pull / Push / Hybrid Model)
- Newsfeed Ranking (Affinity x Weight x Decay)
- Social Graph Design
- Graph Database (Neo4j, ArangoDB)
- Mutual Friends / Recommendation Algorithm

E — Evaluate Trade-offs ★★★★★

- Common Bottleneck Patterns
- SPOF Identification
- Scaling Strategy for Bottlenecks
- Bottleneck Identification & Resolution ★★★★★

D — Done & Extensions ★★★★★

- Revisit Out of Scope items if time permits
- Suggest future improvements
- Confirm design meets all requirements

Common Design Considerations ★★★★★

- Caching Strategy (Redis, Eviction Policy, Cache Miss)
- Data Partitioning Strategy (Sharding, Consistent Hash)
- Database Cleanup Strategy (Active vs Passive)
- Metrics & Analytics Collection
- Security (Private URLs, Authorization, API Gateway)
- Feed Generation (Pull / Push / Hybrid Model)
- Newsfeed Ranking (Affinity x Weight x Decay)

Interview Tips & Best Practices ★★★★★

- Ask Clarifying Questions Before Designing
- Think Out Loud — Explain Reasoning at Every Step
- Start Simple, Then Iterate
- Discuss Trade-offs, Not Just Solutions
- Time Management per Interview Step
- Know When to Go Deep vs Stay High Level
- Handle Ambiguity Gracefully
- Two-Way Interview Communication Strategy
- Avoid Technology Bias
- Read Company Engineering Blog Before Interview
- Be Humble About Knowledge Gaps

Final Review & Reinforcement ★★★★★

- Review All Design Concepts
- Summarize Common Bottlenecks
- Revise Patterns & Architectures
- Deep Dive on Weak Areas

⚠ Most Common Failures

- Memorizing architectures and reciting them — interviewers always catch this
- Being technically strong but unstructured — structured problem-solving separates pass from fail

Frontend System Design

Frontend Architecture Patterns

- Diagramming Tools (Excalidraw, draw.io) ★★★★★
- Frontend HLD vs LLD ★★★★★
- Micro-frontend Architecture ★★★★★
- Module Federation (Webpack 5) ★★★★★
- Monorepo Architecture (Nx, Turborepo) ★★★★★
- Component-Driven Development ★★★★★
- Design System & Component Library Architecture ★★★★★
- Island Architecture (Astro, Fresh) ★★★★★
- Server Components Architecture (Next.js RSC) ★★★★★
- Frontend for AI Applications ★★★★★

Frontend Performance Architecture

- Critical Rendering Path Optimization ★★★★★
- Code Splitting & Lazy Loading ★★★★★
- Bundle Optimization (Tree Shaking, Minification) ★★★★★
- Image Optimization (WebP, AVIF, Responsive Images) ★★★★★
- Resource Hints: Preload, Prefetch, Preconnect ★★★★★
- Web Vitals: LCP, INP, CLS ★★★★★
- Web Vitals Budget & Enforcement ★★★★★
- Virtual Scrolling / Windowing for Large Lists ★★★★★
- Memoization & Re-render Optimization ★★★★★
- Service Worker & Caching Strategy ★★★★★
- Edge Rendering vs SSR vs CSR vs SSG — When to Use ★★★★★

State Management at Scale

- Global State Architecture for Large Apps ★★★★★
- State Normalization ★★★★★
- Optimistic Updates Pattern ★★★★★
- Real-time State Sync (WebSocket + State) ★★★★★
- Offline State Management ★★★★★
- State Machines for Complex UI (XState) ★★★★★

Frontend Caching & Data Fetching

- Browser Cache Headers (Cache-Control, ETag) ★★★★★
- Stale-While-Revalidate (SWR, React Query) ★★★★★
- HTTP Caching Strategies for Frontend ★★★★★
- Data Normalization in Client Cache (Apollo, RTK Query) ★★★★★
- Pagination vs Infinite Scroll — Architecture ★★★★★
- Prefetching & Predictive Loading ★★★★★
- Offset vs Cursor-based Pagination ★★★★★

Scalable Frontend Infrastructure

- CDN for Static Assets ★★★★★
- Edge Functions / Edge Computing for Frontend ★★★★★
- A/B Testing Architecture ★★★★★
- Feature Flags System ★★★★★
- Frontend Observability: Error Tracking (Sentry) ★★★★★
- Real User Monitoring (RUM) ★★★★★
- Synthetic Monitoring ★★★★★
- Frontend Build Pipeline & CI/CD ★★★★★
- Progressive Web Apps (PWA) Architecture ★★★★★

Frontend Security

- XSS (Cross-Site Scripting) Prevention ★★★★★
- CSRF Protection ★★★★★
- Content Security Policy (CSP) ★★★★★
- Secure Storage (localStorage vs sessionStorage vs HttpOnly Cookie) ★★★★★
- OAuth 2.0 / PKCE for SPAs ★★★★★
- Frontend Rate Limiting & Throttling ★★★★★

Frontend System Design Examples

- Design Facebook News Feed Frontend ★★★★★
- Design Google Autocomplete / Search UI ★★★★★
- Design YouTube Frontend Architecture ★★★★★
- Design an Infinite Scroll Component at Scale ★★★★★
- Design a Real-time Collaborative Editor (Google Docs) ★★★★★

- Design a Design System (like Material UI) ★★★★★
- Design an Analytics Dashboard (Real-time) ★★★★★

REDCAAP Framework

RE — Requirements

Functional Requirements

- Core features & user interactions
- User stories & use cases

Non-Functional Requirements

- Performance (latency, load time)
- Accessibility (a11y)
- Scalability & availability
- Compatibility (devices, browsers)
- Security
- Localization / i18n

D — Design Patterns & Architecture Decision

- MVC vs MVVM vs MVP
- SPA vs MPA
- Monolithic SPA vs Micro-frontend
- Component-based vs Monolithic UI
- Unidirectional vs Bidirectional data flow
- Rendering Strategy (CSR/SSR/SSG/ISR/Hybrid)

C — Component & Hierarchy

- Break UI into modular, reusable components
- Single Responsibility per component
- Component hierarchy definition
- State management within components
- Avoid unnecessary re-renders

A — Architecture (HLD)

- Modular frontend components → cohesive UI
- Frontend components → Backend service mapping
- State management layer design
- Routing architecture

- Infrastructure (CDN, CI/CD, Service Workers)

A — API Design

- Choose API style (REST / GraphQL / gRPC / WebSocket / SSE)
- Define endpoints clearly
- Data model design (avoid over/under-fetching)
- Authentication & Authorization
- Error handling patterns
- Pagination, filtering, sorting

P — Performance Optimization

- Core Web Vitals (LCP, FID/INP, CLS)
- Code splitting & lazy loading
- Bundle size optimization & tree shaking
- Image optimization (WebP, AVIF)
- Caching (Browser, CDN, Service Worker)
- Virtualization for long lists
- Debouncing & throttling
- Accessibility (ARIA, semantic HTML, keyboard nav)
- Security (XSS, CSP, CORS, rate limiting)
- Localization & i18n support
- Observability (Sentry, Web Vitals monitoring)

RADIO Framework

R — Requirements Exploration

Functional Requirements

- What are the core user stories?
- Which devices & browsers need support?
- Offline support needed?

Non-Functional Requirements

- Performance targets?
- Accessibility needs?
- SEO requirements?
- Real-time requirements?
- Security constraints?

A — Architecture / High-Level Design

- Application type (SPA / MPA / Micro-frontend)
- Framework choice & rendering strategy
- Component hierarchy overview
- State management strategy
- Routing mechanism
- Infrastructure (CDN, deployment)

D — Data Model (Client-side)

- Data types & shapes (TypeScript)
- Normalization vs nested structures
- Client state structure
- Server state structure (separate from client)
- Caching strategy (React Query / SWR)
- LocalStorage / SessionStorage / IndexedDB use cases

I — Interface Definition (API)

- API style (REST / GraphQL / WebSocket / SSE)
- Endpoint structure & naming
- Request/Response data format
- Authentication (JWT, OAuth)
- Pagination, filtering, sorting
- Error handling patterns
- API versioning

O — Optimizations

- Network: Compression, caching, batch requests
- Rendering: SSR/SSG, lazy loading, hydration
- Images: WebP/AVIF, responsive images, CDN
- Bundle: Code splitting, tree shaking
- Runtime: Memoization, virtualization, debouncing
- Accessibility: Semantic HTML, ARIA, keyboard nav
- Security: XSS, CSP, CORS, rate limiting
- Observability: Core Web Vitals, Sentry, Lighthouse

Frontend System Design Interview Framework

1. Question Types ★★★★★

Type 1 — Application Design

- Use REDCAAP or RADIO framework

- Examples: News Feed, Chat, E-commerce, Video Streaming, Collaborative Editor

Type 2 — UI Component Design

- Use Component-specific approach
- Examples: Autocomplete, Modal, Dropdown, Image Carousel, Rich Text Editor, Kanban Board (Drag & Drop) , Data Table

2. UI Component Design Framework ★★★★★

- Component API (Props) definition
- Subcomponent breakdown
- Internal state design
- Theming & customization
- Keyboard navigation & focus management
- ARIA roles & accessibility
- Edge cases & error states

3. Product-specific Deep Dive Decision ★★★★★

E-commerce (Amazon, Airbnb)

- SEO, Performance, Core Web Vitals

Social Feed (Facebook, Twitter)

- Pagination, Infinite Scroll, Optimistic Updates

Chat / Messaging (WhatsApp, Slack)

- Real-time protocol (WebSocket/SSE)
- Message ordering & delivery guarantees

Video Streaming (YouTube, Netflix)

- Adaptive streaming, Media optimization
- Buffering strategy

Collaborative Apps (Google Docs, Figma)

- Conflict resolution (OT/CRDT)
- Real-time state sync

Maps / Location (Uber, Google Maps)

- Real-time location updates
- Map rendering performance

Photo Sharing (Instagram, Pinterest)

- Media upload & optimization
- Masonry layout, Virtualization